

Phase 3: the bottom bar

BeamMP Race Manager Client: dhardesty99

Five buttons on the bottom bar: Reposition, Spare tire, Repair, Fuel and Lights. Right now they tell you what they would cost and then do nothing. Phase 3 makes them work.

This is a small phase next to phase 2, because most of the hard parts are already built.

1 What already exists

Shipped in phase 1 and 2, so it is not work in this phase:

- The five buttons, with a key bound to each one.
- The rule that the car must be stopped.
- The penalty system. Adding seconds to a run already works.
- The results screen already names each penalty by reason: Recovery, Spare tire, Repair.
- Penalties only exist inside a run. Free driving costs nothing.

What is missing is the action itself and the wait that goes with it.

2 The five buttons

Button	Hold	Penalty	What it does
Reposition	5s	60s	Puts the car back upright and on the road.
Spare tire	30s	30s	Fixes flat tires. See section 9.
Repair	60s	30s	Fixes the whole car.
Fuel +25%	20s	none	Adds a quarter of a tank.
Lights	none	none	Head lights. Works already.

Hold means the car is locked in place for that long while the job is done. **Penalty** means seconds added to your race time at the end.

The race clock keeps running during a hold. So a spare tire in a race costs 30 seconds of waiting and 30 seconds on the clock. One minute in total.

3 Settings you can change

These live on the server, so any number can change without anyone reinstalling the mod.

```
holds = { reposition = 5.0, spareTire = 30.0, repair = 60.0, fuel = 20.0 }  
penalties = { recovery = 60.0, flatTire = 30.0, repair = 30.0, missedGate = 30.0 }  
fuelStep = 0.25
```

4 What happens when you press one

1. The client checks you are in a car and stopped. If not it says so and nothing happens.

2. The client asks the server.
3. The server decides. It checks you are in a run, that you are not already working, and that the button is one it knows.
4. If yes, the server starts the hold and adds the penalty straight away.
5. The client freezes the car and shows a countdown.
6. When the hold ends the server tells the client to do the job. The car is fixed and unfrozen.

The penalty is added at the start, not the end. If it were added at the end you could quit the hold and get the repair for free.

5 The messages

Four messages, in the same envelope everything else already uses.

client to server	service.use	{ which = "repair" }
server to client	service.hold	{ which = "repair", hold = 60, penalty = 30 }
server to client	service.run	{ which = "repair" }
server to client	service.failed	{ which = "fuel", why = "no_tank" }

6 Server side

The server owns the decision, the hold clock and the penalty. This is the whole of it.

```

local jobs = {}

local ACTIONS = {
  reposition = { hold = "reposition", penalty = "recovery" },
  spare      = { hold = "spareTire",   penalty = "flatTire" },
  repair     = { hold = "repair",      penalty = "repair" },
  fuel       = { hold = "fuel",        penalty = nil },
}

function RM.service.use(pid, d)
  local which = type(d) == "table" and tostring(d.which) or ""
  local action = ACTIONS[which]
  if not action then return false, "no_such_action" end
  if jobs[pid] then return false, "already_working" end

  local r = RM.race.get(pid)
  local racing = r ~= nil and r.state == "running"

  local hold = racing and (RM.config.holds[action.hold] or 0) or 0
  local cost = nil
  if racing and action.penalty then cost = RM.config.penalties[action.penalty] end
  if cost and cost > 0 then RM.race.penalty(pid, cost, action.penalty) end

  jobs[pid] = {
    which = which,
    endsAt = RM.now() + hold,
    cost = cost,
    reason = action.penalty,
  }

  RM.bus.queue(pid, "service.hold", { which = which, hold = hold, penalty = cost })
  return true, jobs[pid]
end

function RM.service.tick(now)
  for pid, job in pairs(jobs) do
    if now >= job.endsAt then
      jobs[pid] = nil
      RM.bus.queue(pid, "service.run", { which = job.which })
    end
  end
end

```

```

    end
end

function RM.service.forget(pid)
    jobs[pid] = nil
end

```

RM.service.tick runs on the 100ms tick that already exists, so this adds no new timer and no per frame work.

7 In the game

Every call below is one the game already uses itself. None of it is invented.

```

local function freeze(veh, on)
    core_vehicleBridge.executeAction(veh, "setFreeze", on)
end

local function repairAll(veh)
    veh:queueLuaCommand("beamstate.reset()")
end

local function addFuel(veh, share)
    core_vehicleBridge.requestValue(veh, function(tanks)
        for _, tank in pairs(tanks or {}) do
            if tank.energyType ~= "electricEnergy" then
                local room = tank.maxEnergy - tank.currentEnergy
                local add = math.min(room, tank.maxEnergy * share)
                core_vehicleBridge.executeAction(veh, "setEnergyStorageEnergy",
                    tank.name, tank.currentEnergy + add)
            end
        end
        end, "energyStorage")
    end

local function putBack(veh, gate)
    local dir = vec3(math.cos(gate.yaw), math.sin(gate.yaw), 0)
    spawn.safeTeleport(veh, vec3(gate.pos.x, gate.pos.y, gate.pos.z),
        quatFromDir(dir, vec3(0, 0, 1)))
end

```

spawn.safeTeleport is the same call that already puts you on the grid, so Reposition behaves exactly like the start of a race.

8 The spare tire problem

I checked this in the game files before writing it down, and it is worth saying plainly.

BeamNG can burst a tire on command. It cannot mend one. There is a `deflateTire` and there is no matching call to put a tire back. The only repair the game offers is `beamstate.reset`, which fixes the whole car.

So Spare tire cannot be cheaper and smaller than Repair by itself. Left alone it would be the same job for half the wait, and nobody would ever press Repair.

The game does record which tire is flat, so this is checkable:

```
wheels.wheels[id].isTireDeflated
```

Three ways to handle it. My default is the third.

1. Drop Spare tire and keep Repair only.

2. Keep both, and accept Spare tire is the cheaper full repair.
3. Keep both, and only let Spare tire be pressed when a tire is actually flat. It still fixes the whole car, but you cannot use it as a cheap repair because you need a flat first.

9 Rules and edge cases

- You cannot press a second button while a hold is running.
- You cannot cancel a hold. The penalty is already counted.
- Leaving the server during a hold ends the run the normal way, as a did not finish.
- If the game cannot do the job, for example fuel on an electric car, the player is told plainly and the penalty is given back.
- Outside a run there is no penalty and no hold. The buttons just work.
- Lights need no hold and no stop, so they stay instant.

10 On screen

- A countdown while the hold runs, under the race clock.
- The clock keeps counting during the hold, because it is still costing you time.
- The penalty shows next to the clock the moment it lands, the same as a missed checkpoint does now.
- The buttons grey out while a hold is running.

11 Questions, and what I will do if you say nothing

Answer with numbers only if you like. Anything you skip gets the default.

	Question	Default if you say nothing
1	Does the hold count on top of the penalty, or instead of it?	On top. The hold is the wait, the penalty is the punishment.
2	Where does Reposition put the car?	Back at the last checkpoint you passed, up-right and facing the right way.
3	How should Spare tire work, given the game cannot mend one tire?	Option 3 in section 9. Only usable when a tire is actually flat.
4	Is there a limit on how many times per run?	No limit. The penalties are the limit.
5	Should Repair be allowed during a race at all?	Yes, at 60 seconds of waiting and 30 on the clock.
6	Do holds apply outside a race?	No. Free driving costs nothing and waits for nothing.
7	Are the numbers in section 3 right?	Yes, and any of them can change later in one line.
8	There is a 37 mph speed limit sitting in the settings that nothing uses. Was that meant to be a pit lane or service area limit?	Leave it alone and ask again in a later phase.

12 Order of work

1. Server side first: the hold, the checks, the penalty. All of it testable without the game.
2. The four actions in the game, easiest first: fuel, repair, spare tire, reposition.
3. The countdown and the greyed out buttons.
4. Testing in game, one button at a time, in a race and out of one.

Server first means most of the phase is proven before the game is opened. That is what worked in phase 2.

13 Testing

- Mock tests for the hold: it starts, it blocks a second press, it ends on time, and the penalty is added once and not twice.
- A test that a run ending badly leaves no hold stuck behind.
- In game, each button twice: once in a race and once in free driving.
- One run using every button, then read the results and check every penalty is listed with the right reason.

14 What could go wrong

- **Fuel differs by vehicle.** Electric cars have no fuel tank. Those get told plainly instead of failing in silence.
- **Freezing a car in multiplayer.** Other players should see the car stopped, which is correct, but it needs checking with two drivers rather than assuming.
- **Reposition on a course that folds back on itself.** The car goes back to the last checkpoint passed, which on a tight loop can be close to a different part of the track. Worth watching on the Baja 1000.

15 Not in this phase

Qualifying and Race on a shared grid are phase 4. Records and personal bests are phase 5. CoPilot and Challenges are phase 6.

Phase 2 is delivered and running on the server. Phase 3 starts on your word, and the eight questions above are the only thing that would change the shape of it.